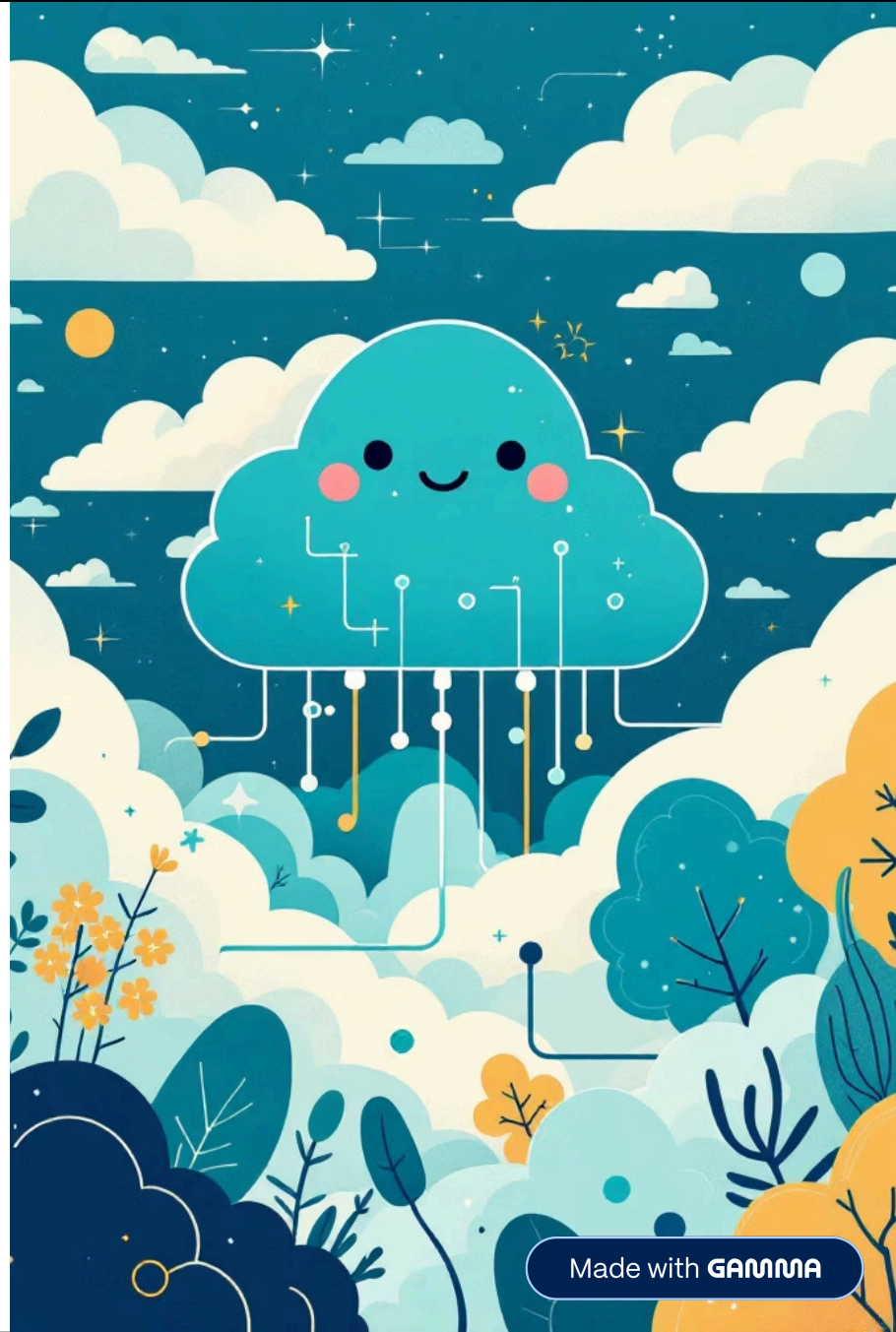
gRPC: Google Remote Procedure Call

- Gabriel Victor Couto Martins de Paula
- Igor de Carvalho Sena Horta
- Matheus Mestre Picerne



Made with GAMMA

A Era Pré-gRPC: RPC nas Décadas de 1990 e 2000

Antes da ascensão do gRPC, a Comunicação de Procedimento Remoto (RPC) já era um pilar fundamental para sistemas distribuídos. Permitia que programas interagissem como se estivessem no mesmo processo, abstraindo a complexidade da comunicação de rede.

Tecnologias Prevalentes

CORBA, DCOM e, posteriormente, SOAP/XML-RPC dominavam o cenário, oferecendo frameworks para construir serviços distribuídos.

Desafios em Interoperabilidade

Essas tecnologias frequentemente sofriam com a complexidade de configuração, incompatibilidade entre linguagens e plataformas, e formatos de serialização ineficientes (como o XML verboso).

Performance e Escalabilidade

A sobrecarga de processamento e a lentidão na serialização e desserialização de dados limitavam a performance, dificultando a escalabilidade em grandes sistemas.

O cenário era de constante busca por soluções mais eficientes e interoperáveis, que pudessem lidar com as crescentes demandas dos sistemas distribuídos.

O Contexto Google: Por Que o gRPC Surgiu?

No início dos anos 2000, o Google enfrentava desafios massivos com a arquitetura de seus sistemas distribuídos. As soluções RPC existentes não eram eficientes para a escala e diversidade de linguagens que a empresa precisava.

Limitações de Escala

As arquiteturas monolíticas e protocolos de comunicação tradicionais não suportavam o crescimento exponencial dos serviços.

Ineficiência de Código

A serialização de dados e o uso da rede eram ineficientes, gerando latência e alto consumo de recursos.

Heterogeneidade

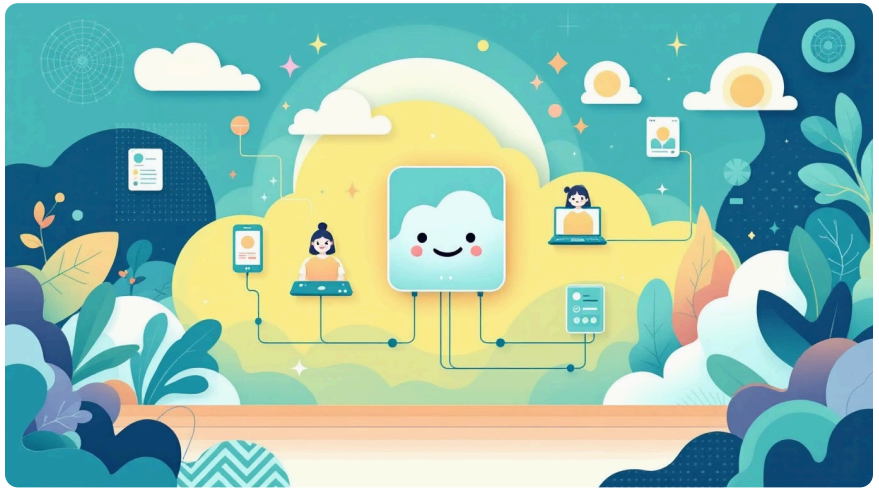
A comunicação entre serviços escritos em diferentes linguagens (C++, Java, Python) era complexa e inconsistente.

Essas dores impulsionaram a criação de uma solução robusta e de alta performance: o gRPC.

O que é gRPC?

gRPC (Google Remote Procedure Call) é um framework moderno de comunicação entre serviços que permite que aplicações distribuídas se comuniquem de forma eficiente como se estivessem chamando funções locais.

Ele combina a velocidade do HTTP/2 com a eficiência da serialização binária, criando um padrão robusto para microserviços.



HTTP/2

Protocolo de transporte moderno com multiplexação e streams bidirecionais

Protocol Buffers

Serialização binária eficiente com contratos fortemente tipados

RPC

Invocação remota de procedimentos como se fossem chamadas locais

Por que usar gRPC?

Em arquiteturas modernas baseadas em microsserviços, a comunicação entre serviços é um ponto crítico de performance e confiabilidade. O gRPC surge como alternativa ao REST tradicional quando há necessidade de características específicas:

Alta Performance

Serialização binária e multiplexação do HTTP/2 reduzem drasticamente o overhead de comunicação

Comunicação Binária

Protocol Buffers geram mensagens compactas e eficientes para transmissão de dados

Contratos Tipados

Definição clara de mensagens e serviços elimina ambiguidades e reduz erros

Streaming de Dados

Suporte nativo para streaming bidirecional, ideal para transmissão contínua de informações

Made with GAMMA

Quando usar gRPC?

O gRPC se destaca em cenários específicos onde performance, eficiência e interoperabilidade são cruciais. Deve ser considerado em contextos como:

Microsserviços e Sistemas Distribuídos

Comunicação interna de alta performance e baixa latência entre serviços em uma arquitetura distribuída.

Ambientes Poliglotas

Integração de serviços desenvolvidos em diferentes linguagens, graças à geração de código multi-linguagem.

Streaming e Redes Restritas

Aplicações que exigem comunicação em tempo real, streaming bidirecional (IoT) e eficiência em redes de baixa largura de banda (mobile).

Como usar gRPC?

A implementação do gRPC segue um padrão de trabalho bem definido que facilita a criação de contratos e geração de código:

Definir o Contrato

Criar arquivo **.proto** descrevendo serviços, mensagens e métodos disponíveis

Gerar Código

Usar compilador protoc para gerar stubs automaticamente para client e server em múltiplas linguagens

Implementar Servidor

Desenvolver a lógica de negócio nos métodos definidos no contrato

Consumir via Cliente

Utilizar o stub gerado para fazer chamadas remotas como se fossem locais

Exemplo de Contrato

Proto

```
syntax = "proto3";

service UserService {
 rpc GetUser (UserRequest) returns (UserResponse);
}

message UserRequest {
 int32 id = 1;
}

message UserResponse {
 string name = 1;
}
```

Desafios e Considerações

Apesar dos benefícios, o gRPC apresenta alguns desafios que devem ser considerados na escolha da tecnologia:

Curva de Aprendizado

Requer compreensão de Protocol Buffers e conceitos de streaming, mais complexo que REST tradicional

Debug Mais Complexo

Mensagens binárias não são legíveis como JSON, exigindo ferramentas específicas para inspeção

Observabilidade

Monitoramento e tracing requerem instrumentação adicional e ferramentas compatíveis

Compatibilidade Browser

Não funciona diretamente em browsers, necessita gRPC-Web ou proxy para interface web

Made with GAMMA

gRPC vs. REST vs. GraphQL: Uma Comparação

A escolha do protocolo de comunicação certo é crucial para a arquitetura de microsserviços. Vamos comparar o gRPC com as alternativas populares, REST/HTTP e GraphQL, em aspectos chave:

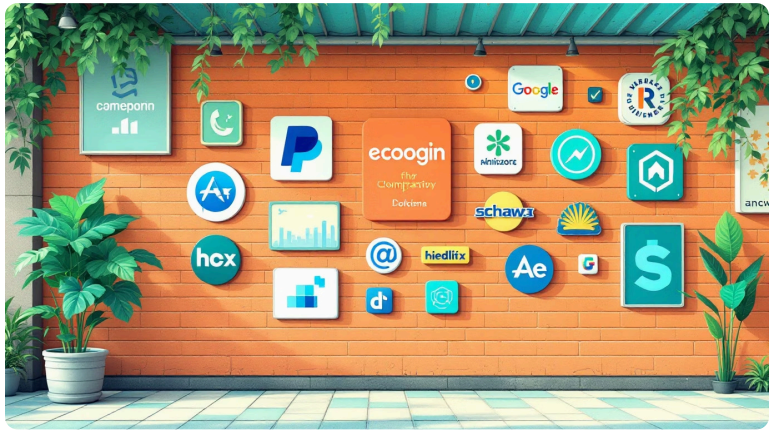
Característica	gRPC	REST/HTTP	GraphQL
Curva de Aprendizado	Alta (Protocol Buffers, HTTP/2, conceitos de streaming)	Baixa (HTTP padrão, JSON, conceitos RESTful)	Média (linguagem de query, schema, resolvers)
Documentação	Gerada automaticamente a partir do arquivo `.proto`	Swagger/OpenAPI, Postman, documentação manual	Introspection de schema, GraphiQL/Playground
Flexibilidade de Queries	Fixa (métodos RPC predefinidos no contrato)	Fixa (endpoints e estrutura de recursos definidos)	Alta (cliente define campos e estrutura da resposta)
Formato de Dados	Protocol Buffers (binário, compacto, eficiente)	JSON, XML (texto, legível)	JSON (texto, cliente customiza)
Suporte a Cache	Difícil (natureza de streaming e formato binário)	Excelente (verbos HTTP e cabeçalhos de cache nativos)	Desafiador (requisições POST comuns, exige cache no cliente ou intermediário)

Cada tecnologia possui seus pontos fortes e fracos, e a melhor escolha depende dos requisitos específicos do projeto, como performance, simplicidade e flexibilidade de consulta.

Cases de Uso e Empresas

Principais Cenários

- Comunicação entre microsserviços em larga escala
- Sistemas de alta performance como fintechs, jogos e machine learning
- Comunicação interna entre backend e serviços internos
- Infraestrutura distribuída e service mesh



Google

Criador e principal mantenedor do gRPC

Netflix

Usa para comunicação interna entre microserviços

Square

Implementação em sistemas de pagamento de alta escala

Dropbox

Utiliza para sincronização eficiente de dados

Outras Implementações RPC

Embora o gRPC seja uma solução robusta, o ecossistema de RPC oferece diversas outras implementações, cada uma com características e otimizações específicas para diferentes contextos.

Twirp

Desenvolvido pelo Twitch, o Twirp é uma implementação RPC leve que prioriza a simplicidade e a familiaridade com HTTP 1.1, utilizando Protocol Buffers. É uma ótima alternativa para projetos que buscam uma infraestrutura mais tradicional, sem a complexidade do HTTP/2, mas ainda se beneficiando da tipagem forte dos Protocol Buffers. https://twitchtv.github.io/twirp/docs/intro.html

Apache Thrift

Um framework RPC multiplataforma e multiprotocolo. O Thrift permite que serviços desenvolvidos em diferentes linguagens (C++, Java, Python, Go, etc.) se comuniquem de forma eficiente, oferecendo seu próprio mecanismo de serialização e transporte. É amplamente utilizado em grandes sistemas distribuídos. https://thrift.apache.org/

Cap'n Proto

Conhecido por sua performance extrema, o Cap'n Proto foca na serialização de dados de zero-copy. Diferente do gRPC e Thrift, ele não exige uma fase de "deserialização", lendo dados diretamente da memória, o que o torna ideal para aplicações com requisitos rigorosos de baixa latência e alta vazão. https://capnproto.org/

Curiosidade: O 'g' em gRPC

Uma peculiaridade divertida do gRPC é que a letra 'g' em seu nome não representa algo fixo. A cada nova versão, a comunidade do gRPC define um novo significado para o 'g', muitas vezes com um toque de humor ou relevância cultural.
https://github.com/grpc/grpc/blob/master/doc/g_stands_for.md?ref=blog.lsantos.dev
O gRPC é mantido pela Cloud Native Computing Foundation https://www.cncf.io/

Primeiras versões

- 1.0 'g' stands for 'gRPC'
- 1.1 'g' stands for 'good'
- 1.2 'g' stands for 'green'
- 1.3 'g' stands for 'gentle'
- 1.4 'g' stands for 'gregarious'

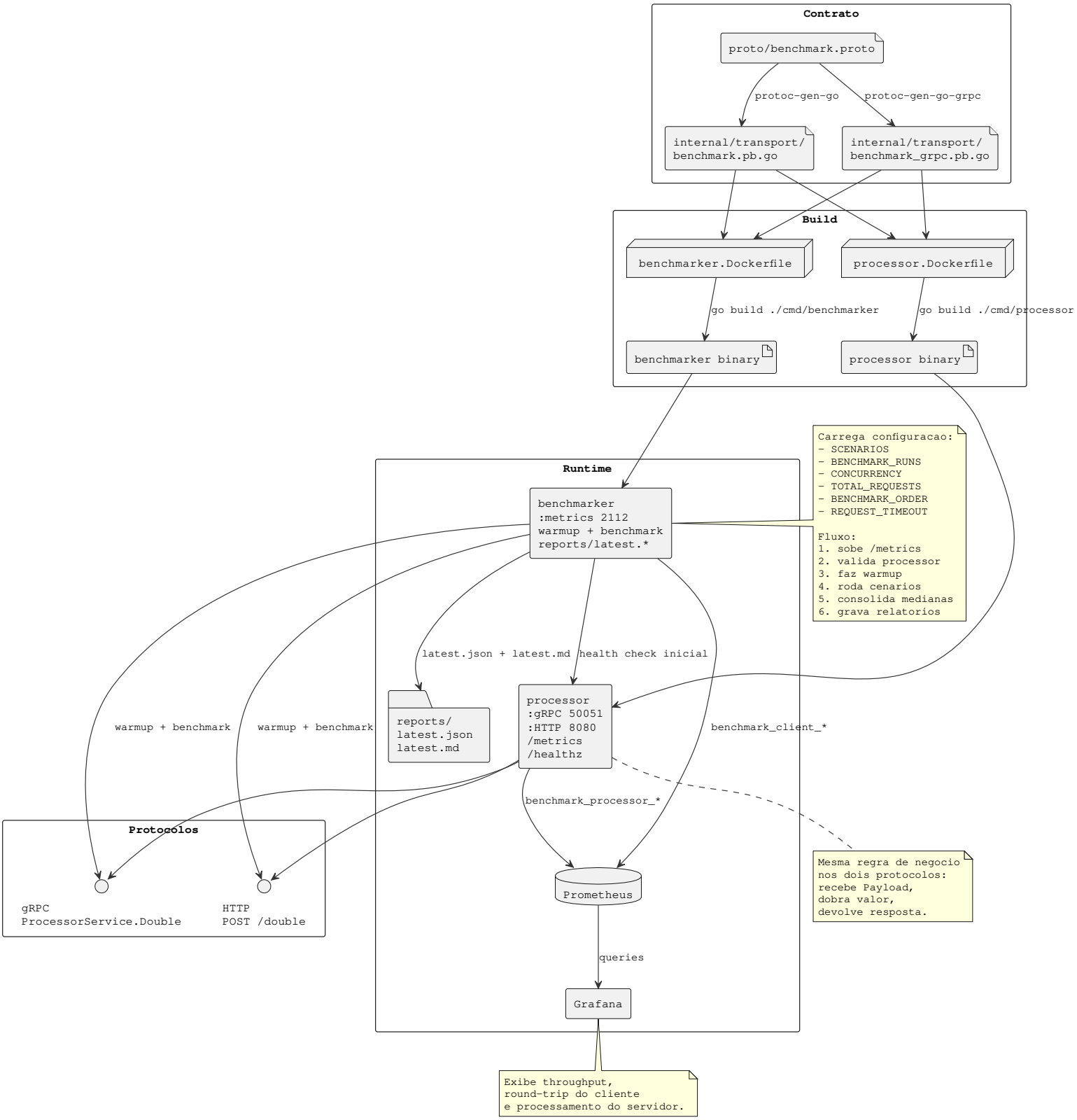
Nomes memoráveis

- 1.21 'g' stands for 'gandalf'
- 1.36 'g' stands for 'gummybear'
- 1.41 'g' stands for 'goat'
- 1.49 'g' stands for 'gamma'
- 1.58 'g' stands for 'goku'

Versões recentes

- 1.70 'g' stands for 'genius'
- 1.71 'g' stands for 'gears'
- 1.73 'g' stands for 'gradient'
- 1.75 'g' stands for 'gemini'
- 1.81 'g' stands for 'graphic'

Ciclo de Vida do Benchmark gRPC vs HTTP JSON



Hands On

Conclusão:

DEPENDE

A escolha entre gRPC e REST não é uma questão de superioridade técnica, mas sim de alinhar as ferramentas aos requisitos específicos do seu sistema. Cada arquitetura impõe trade-offs que moldam a longevidade e a escalabilidade da sua aplicação.

Complexidade vs. Performance

gRPC oferece alta performance, mas introduz complexidade na implementação e infraestrutura de rede.

Observabilidade vs. Eficiência

O ecossistema HTTP/REST é nativamente mais fácil de monitorar e depurar do que o tráfego binário do gRPC.

Curva de Aprendizado vs. Escala

REST possui adoção universal, enquanto o gRPC exige que a equipe domine Protobuf e HTTP/2 para extrair escala real.

Compatibilidade vs. Velocidade

REST garante interoperabilidade com navegadores e firewalls legados, enquanto gRPC prioriza a velocidade entre microserviços.

Não existe uma solução universal. A "melhor" escolha é aquela que equilibra as demandas técnicas do seu projeto com a maturidade operacional do seu time.

Referências

Livros e Documentação Oficial

KLEPPMANN, Martin. *Designing Data-Intensive Applications*: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. Sebastopol: O'Reilly Media, 2017.

GULATI, Satnam; **HILBERT, Stephan**. *gRPC Up and Running*: Building Cloud Native Applications with gRPC, Protobuf, and Go. Sebastopol: O'Reilly Media, 2020.

GOLANG. *gRPC*: A high performance, open source universal RPC framework. Disponível em: **https://grpc.io/docs/what-is-grpc/**. Acesso em: 15 mai. 2024.

CLOUD NATIVE COMPUTING FOUNDATION. *Projects*. Disponível em: **https://www.cncf.io/projects/**. Acesso em: 15 mai. 2024.

GOOGLE. *Protocol Buffers*: Google's language-neutral, platform-neutral, extensible mechanism for serializing structured data. Disponível em: **https://developers.google.com/protocol-buffers/**. Acesso em: 15 mai. 2024.

Implementações RPC e Recursos Adicionais

TWITCH. *Twirp*: A simple framework for service-to-service communication. Disponível em: **https://twitchtv.github.io/twirp/docs/intro.html**.

APACHE SOFTWARE FOUNDATION. *Apache Thrift*. Disponível em: **https://thrift.apache.org/**.

CAP'N PROTO. *Cap'n Proto*. Disponível em: **https://capnproto.org/**.

GRPC. *The 'g' in gRPC*. Disponível em: **https://github.com/grpc/grpc/blob/master/doc/g_stands_for.md**.

CLOUD NATIVE COMPUTING FOUNDATION. *About*. Disponível em: **https://www.cncf.io/**.